Alexandria University | Faculty of Engineering | CCE: Computer Vision

Assignment 1 — Part 2

Frequencies, Pyramids and Blending

TO COMPLETE: team members (first page of the final PDF)

Replace with the names and IDs of the three team members.

Member 1: `[name]` ID: `[id]`
Member 2: `[name]` ID: `[id]`
Member 3: `[name]` ID: `[id]`

Results at a glance

Section	Main result
2.1 Hybrid images	$(\sigma_{\text{low}}, \sigma_{\text{high}})$: man–cat $(10,5)$; pair 2 $(6,3)$; pair 3 $(6,3)$; pair 4 $(6,3)$ — pair 4 (front face against profile) fails, as expected.
2.2 Aliasing	For a $4\times$ shrink of `rings.png` the smallest σ with no visible false circles is $\sigma = 3$ (at $\sigma = 2.5$ a faint residue is left at the borders).
2.3 Pyramids	Rebuild error $\max \lvert I - \hat{I} \rvert = 1.11 \times 10^{-16}$ (machine precision); the six-level Gaussian pyramid holds $1.3336\times$ the pixels of the image ($\to 4/3$).
2.4 Blending	Oraple: left-half mask, 6 levels. Face on a pumpkin: traced-outline mask (14.9% of the image), 5 levels. Eye on the moon: traced-outline mask (5.8%), 5 levels.

Conventions used throughout. All blurs run on our own separable convolution with reflect padding (`np.pad(..., mode="reflect")`), as the handout requires from Part 2 onward. Images are float64 in $[0,1]$; results are clipped to $[0,1]$ only for display. Signed images (high-pass, Laplacian levels) are shown with zero as mid-gray. Our own images were resized so that the longer side is at most 1000 pixels. Every section below has the same shape: what we did, what came out, the *answer* to the handout's question, and a short reflection on *why* the result looks the way it does.

1 Hybrid images (2.1)

1.1 Method

A hybrid keeps the low frequencies of an image A and the high frequencies of an image B:

$$H = \underbrace{G_{\sigma_{\text{low}}} * A}_{\text{low-pass of } A} + \underbrace{\left(B - G_{\sigma_{\text{high}}} * B\right)}_{\text{high-pass of } B}, \tag{1}$$

where G_σ is a Gaussian of standard deviation σ pixels. B is first warped onto A with `align_images.py` using two matching points (the eyes for the faces). The computation is done on grayscale images, as in the handout figure.

A Gaussian of standard deviation σ has frequency response $e^{-2\pi^2\sigma^2 f^2}$. We call its *cutoff* the frequency where the response drops to $e^{-1/2} \approx 0.61$:

$$f_c = \frac{1}{2\pi\sigma} \quad \text{cycles per pixel.} \tag{2}$$

Larger σ_{low} means a blurrier low image (lower cutoff); larger σ_{high} means the high-pass keeps *less* (its cutoff moves down).

```python
def make_hybrid(im_low, im_high, sigma_low, sigma_high, clip=True):
    low = gaussian_blur(im_low, sigma_low)
    high = im_high - gaussian_blur(im_high, sigma_high)
    hybrid = low + high
    return {..., "hybrid": np.clip(hybrid, 0, 1) if clip else hybrid}
```

1.2 The three hybrids and their σ values

Pair	Low-pass from / High-pass from	σ_{low}	σ_{high}	f_c^{low}	f_c^{high}	Outcome
man–cat (starter)	man / cat	10	5	0.016	0.032	works
pair 2 (ours)	face A / face B	6	3	0.027	0.053	partial
pair 3 (ours)	smiling / serious (same person)	6	3	0.027	0.053	works well
pair 4 (ours)	front view / profile view	6	3	0.027	0.053	fails

In all four pairs $\sigma_{\text{low}} = 2\sigma_{\text{high}}$, so the high-pass starts one octave above where the low-pass ends. Frequencies in that gap are weak in *both* layers, which is what keeps the two readings apart.

1.3 Favorite pair: man and cat, the whole process

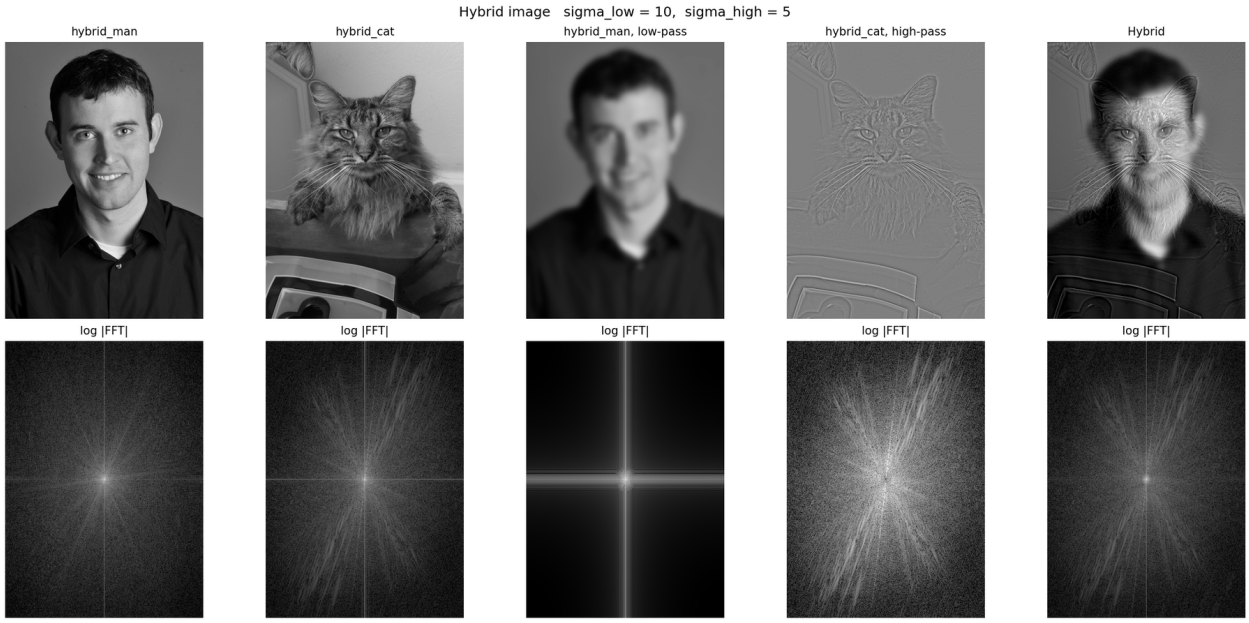


Figure 1: Man (low-pass, $\sigma = 10$) plus cat (high-pass, $\sigma = 5$). Top: the two inputs, the two filtered images and the hybrid. Bottom: $\log|\text{FFT}|$ of each, computed on the grayscale images with `np.log(np.abs(np.fft.fftshift(np.fft.fft2(gray))))`.

Why it looks like this

Reading the spectra. (i) The man's low-pass spectrum is bright only close to the center and black elsewhere: everything above roughly 0.016 cycles/pixel was removed. (ii) The cat's high-pass spectrum is the opposite: a dark spot at the center (the low frequencies are gone) and energy spread over the whole plane. The streaks follow the orientation of the fur and the whiskers. (iii) The hybrid's spectrum looks like the man's near the center and like the cat's far from it, because the Fourier transform is linear: $\mathcal{F}\{H\} = \mathcal{F}\{\text{low}\} + \mathcal{F}\{\text{high}\}$, and in each frequency region only one of the two terms is large. (iv) The bright horizontal and vertical cross in the blurred spectrum is not part of the picture: the DFT treats the

image as periodic, so the mismatch between opposite borders acts like a sharp edge and leaks energy along the two axes.

Why the man is the low image and the cat the high one. The man's face is smooth, so its identity lives in coarse shapes (outline of the head, position of the eyes) that survive blurring. The cat's identity lives in fur and whiskers, which are high-frequency texture. Swapping the roles would put a blurry cat under a face-sized pattern of skin pores.

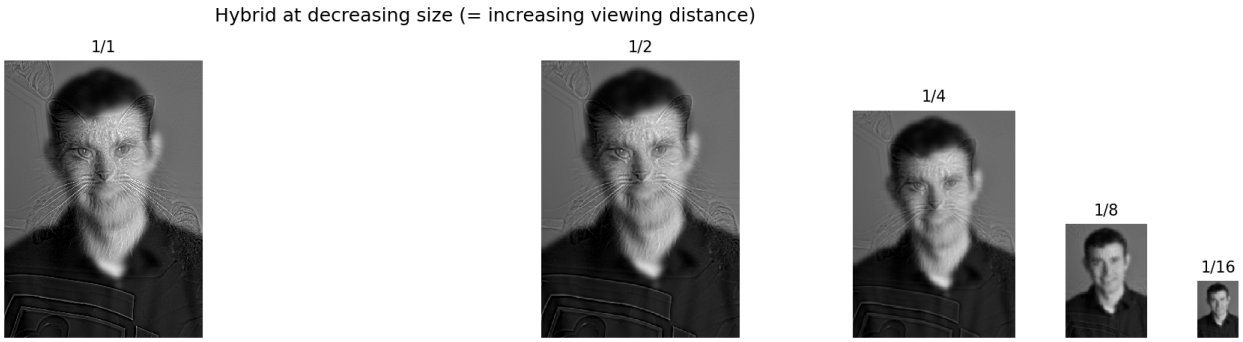


Figure 2: The hybrid shown at 1, $1/2$, $1/4$, $1/8$ and $1/16$ of its size, each tile drawn at its true pixel size (a stand-in for increasing viewing distance). The cat dominates at 1 and $1/2$; at $1/8$ and $1/16$ only the man remains.

Why it looks like this

Why identity changes with distance. Shrinking an image by a factor s multiplies all its spatial frequencies (in cycles per pixel) by s, and the blur that precedes subsampling removes whatever no longer fits. The cat's fur, with features a few pixels wide, is the first thing to disappear; the man's head, whose features are tens of pixels wide, is still resolved at $1/8$. The eye does the same thing as we step back: it acts as a low-pass filter whose cutoff, measured in image coordinates, falls with distance.

Tuning σ_{low} and σ_{high}. Figure 3 varies both. Reading across, a larger σ_{low} blurs the man further, until at 16 he is a dark halo behind the cat. Reading down, a larger σ_{high} lowers the cat's cutoff, so more mid-frequency cat gets in, until at 9 the cat is everywhere and the man is hard to find even from afar. The balance is best around $(10, 5)$: close up the cat is clear, from far away the man is clear, and neither layer hides the other.

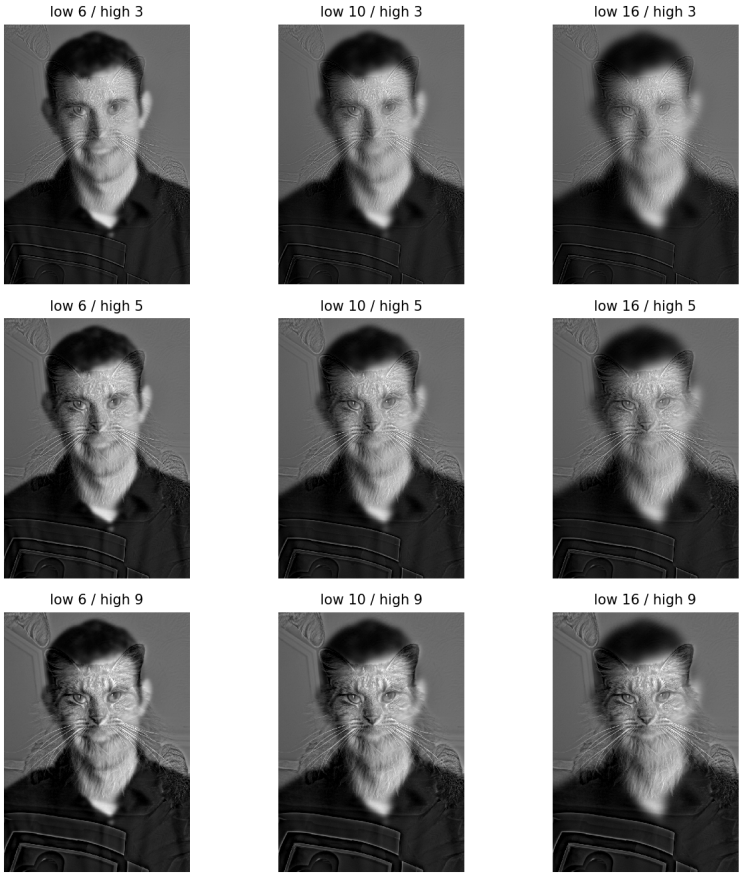


Figure 3: Man–cat for $\sigma_{\text{low}} \in \{6, 10, 16\}$ (columns) and $\sigma_{\text{high}} \in \{3, 5, 9\}$ (rows).

1.4 Our own pairs

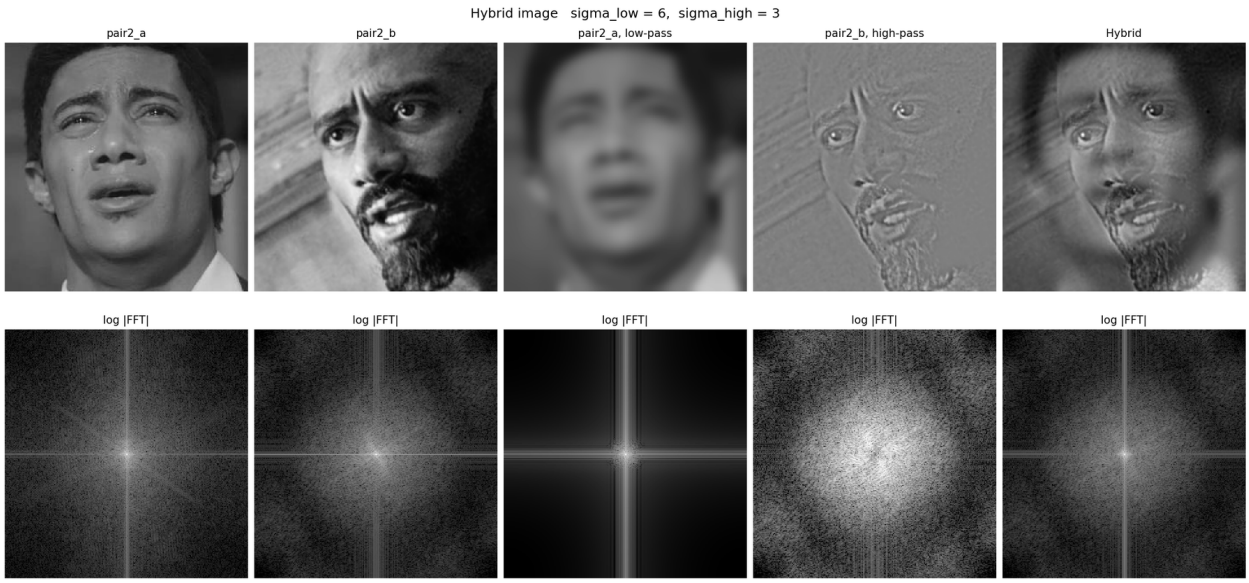


Figure 4: Pair 2, $\sigma_{\text{low}} = 6$, $\sigma_{\text{high}} = 3$.

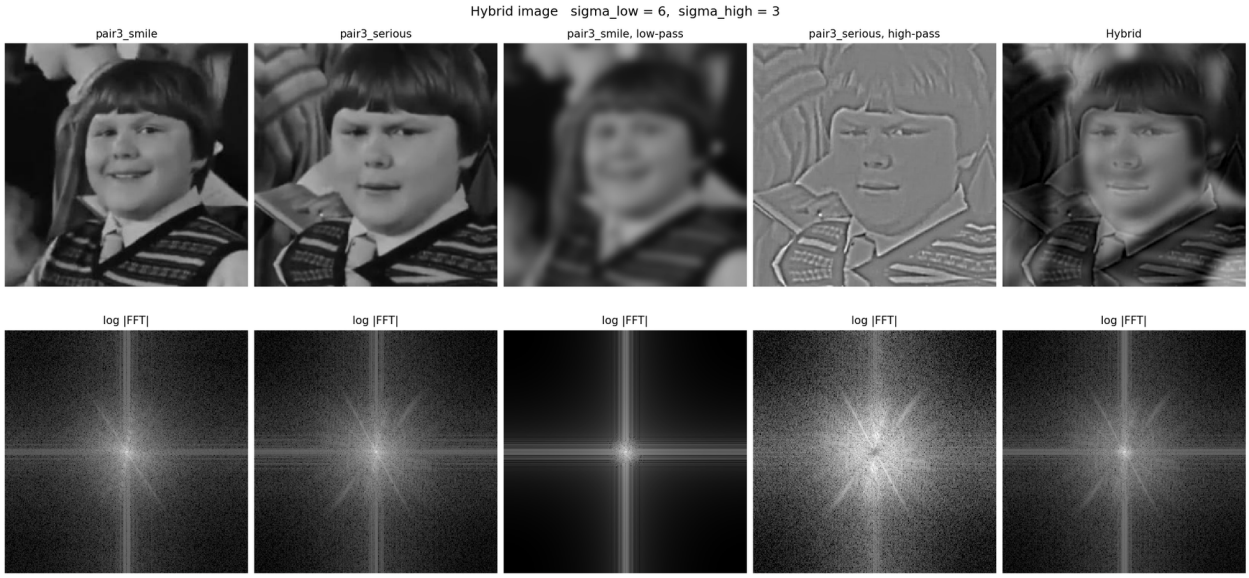


Figure 5: Pair 3 (same person, smiling and serious), $\sigma_{\text{low}} = 6$, $\sigma_{\text{high}} = 3$.

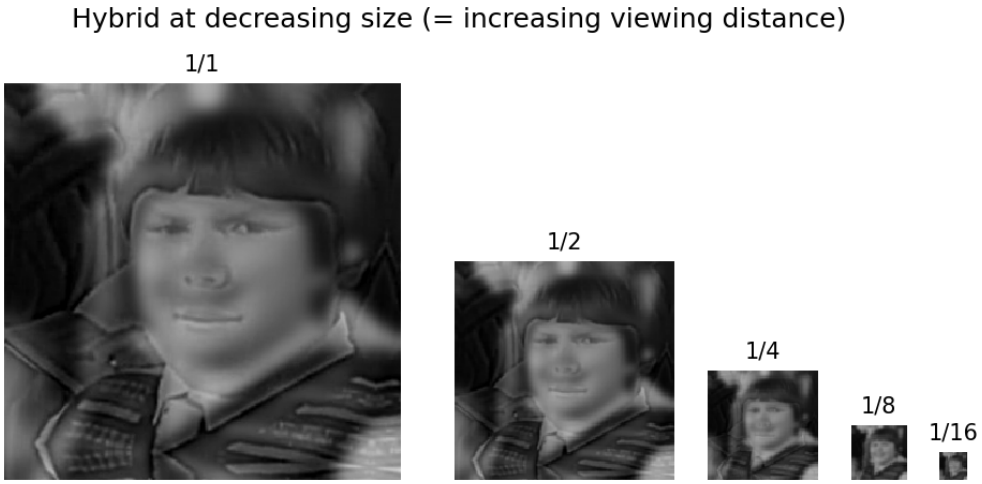


Figure 6: Pair 3 at decreasing size.

Why it looks like this

Pair 2 (partial). The two eyes line up, because the eyes are the two alignment points. Away from the eyes nothing is forced to match: the second head is tilted and its mouth and beard land on the first face's cheeks and chin, so the hybrid reads as a face with an extra overlaid mouth rather than as one face. A two-point alignment is a similarity transform (rotation, uniform scale, shift) and cannot correct a different head pose.

Pair 3 (works best structurally). The same person in the same pose: after the eyes are aligned, the brows, nose, mouth and hairline of the two images already coincide, so the two layers fuse into one face. This is the ideal use of the method: the high-pass layer re-draws the edges of the mouth and brows on top of the low-pass face, and the two layers disagree only about the expression.

1.5 The failing pair: front view against profile (pair 4)

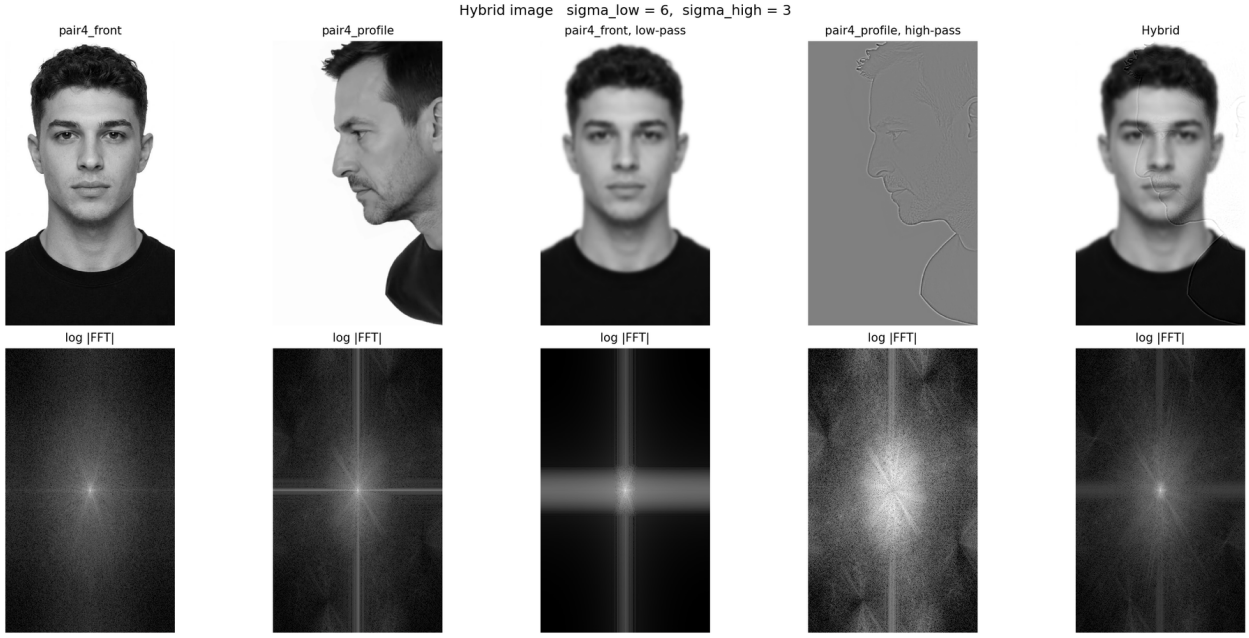


Figure 7: Pair 4: frontal face (low-pass) and profile (high-pass). The hybrid is essentially the frontal face with a few stray contour lines of the profile.

Answer

Why does pair 4 fail? Hybrid images need the two layers to be *the same shape seen at two scales*. A front view and a side view are different shapes: the nose, the lips and the ear are in different places, and the profile face is cut by the image border. Two matching points can only rotate, scale and translate the second image; they cannot turn a head. After the best possible alignment, the profile's edges do not sit on anything meaningful in the front face, so they are read as scribbles, not as a second face.

Why it looks like this

The profile's high-pass is also nearly empty: the photo has a plain white background and smooth skin, so the only high-frequency content is the thin silhouette line (see its spectrum, which is concentrated around the center). The hybrid therefore adds almost nothing to the frontal face except that outline. It is a useful negative result: *alignment of a few points is necessary for a hybrid, but it is the similarity of the two images' structure that makes it work.*

2 Aliasing (2.2)

2.1 Method

Keeping every k-th pixel samples the image at $1/k$ samples per pixel, so the highest frequency it can still represent (the new Nyquist limit) is

$$f_N = \frac{1}{2k} \text{ cycles per } \textit{original} \text{ pixel} \qquad (f_N = 0.125 \text{ for } k = 4). \tag{3}$$

Any component with $f > f_N$ is not lost: it *folds* to a lower frequency $f' = |f - m/k|$ (m the nearest integer) and is indistinguishable from a genuine low-frequency component. Equivalently, the spectrum of the subsampled image is the original spectrum copied at multiples of $1/k$; where the copies overlap, high-frequency energy lands in the low-frequency band. Blurring first removes the energy above f_N *before* the copies are made.

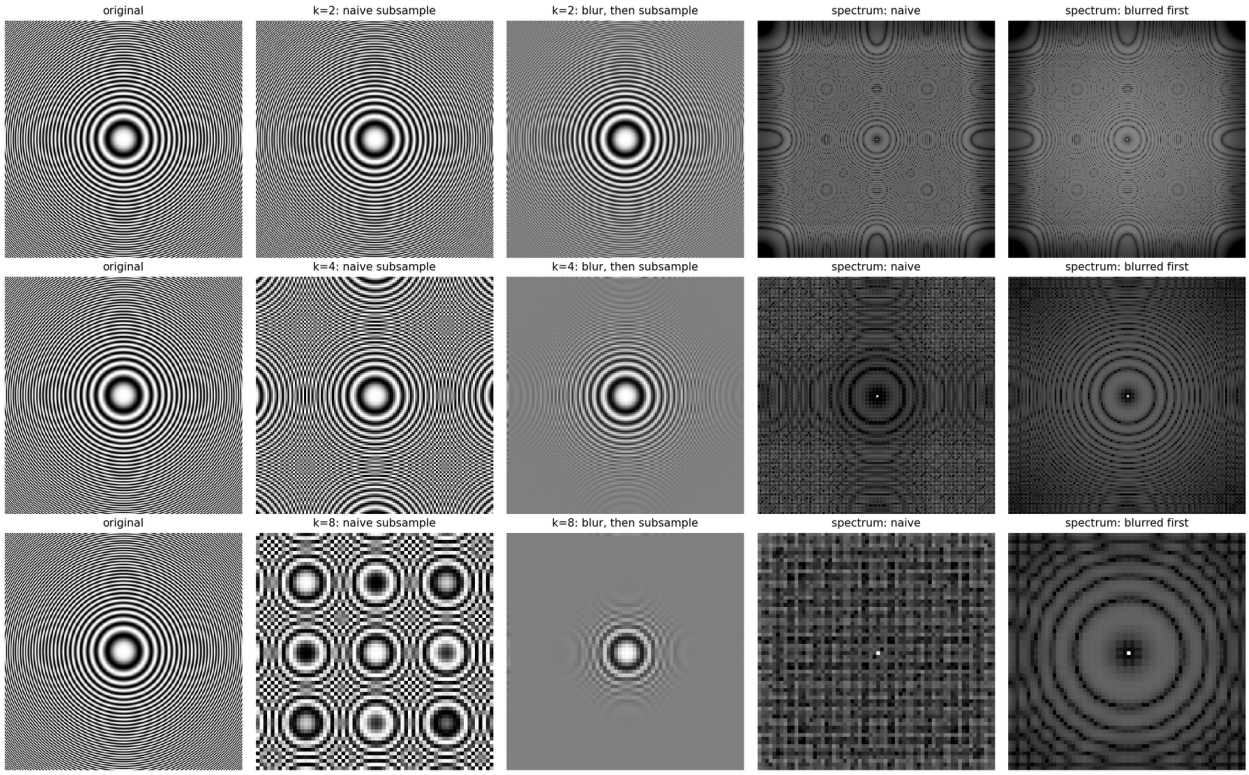


Figure 8: `rings.png` shrunk by $k = 2, 4, 8$: original, naive subsample, blur-then-subsample (with $\sigma = k/2$), and the spectrum of each. The shrunk images are blown back up to full size with nearest-neighbour pixels. Naive subsampling creates extra circle centers (moiré); blurring first does not.

2.2 The $4\times$ shrink of `rings.png`

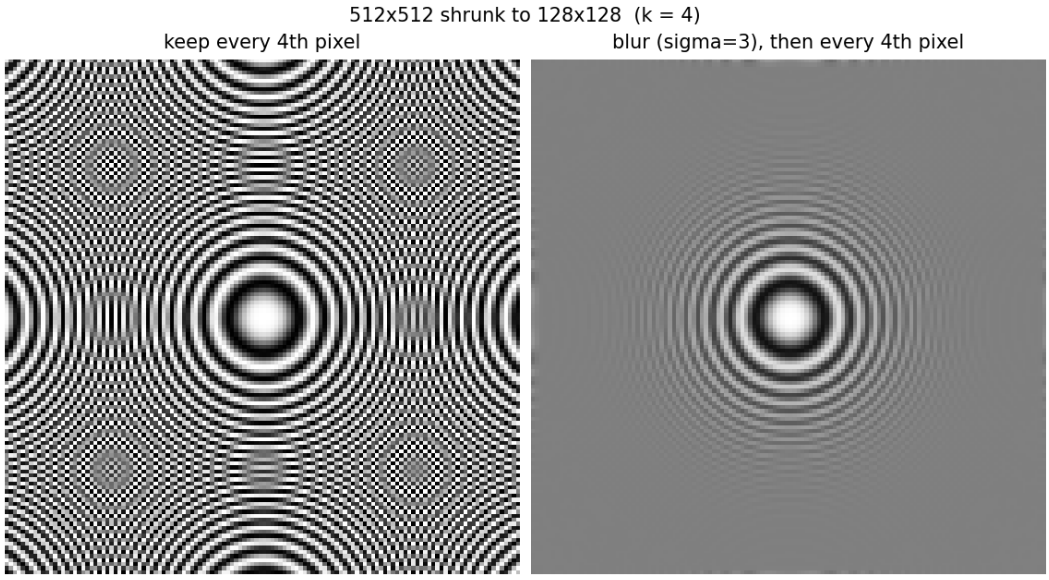


Figure 9: The two quarter-size results side by side ($512 \times 512 \to 128 \times 128$): left, keeping every 4th pixel; right, Gaussian blur with $\sigma = 3$ and then every 4th pixel.

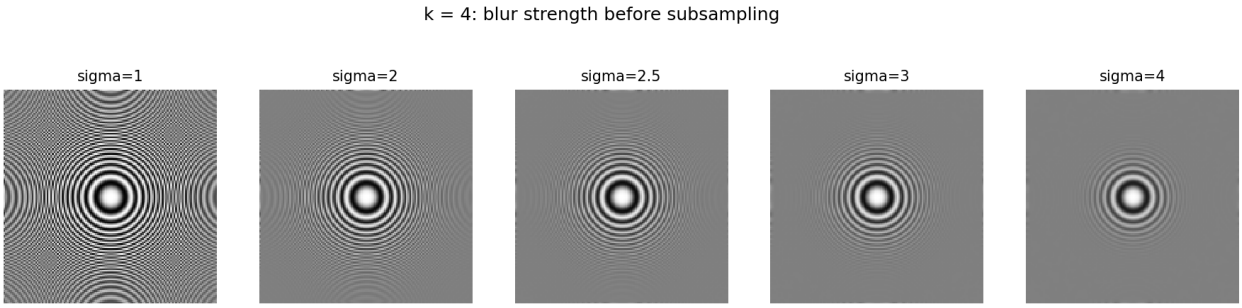


Figure 10: Blur strength before subsampling at $k = 4$: $\sigma = 1, 2, 2.5, 3, 4$ (shown at $4\times$ zoom).

To put a number next to the eye test: the outer rings of the original are far too fine to survive a $4\times$ shrink, so a correctly anti-aliased result is flat gray there. Any contrast left in that region is false circles. Table 1 gives the standard deviation of the outer region (radius above 35% of the image side) next to the Gaussian's response at the new Nyquist limit, $H(f_N) = e^{-\pi^2\sigma^2/(2k^2)}$.

	naive	σ=1	1.5	2	2.5	3	3.5	4
outer-ring std	0.353	0.167	0.075	0.028	0.011	0.006	0.005	0.004
$H(f_N)$	—	0.735	0.50	0.29	0.15	0.06	0.02	0.007

Table 1: Leftover false-circle contrast against blur strength, $k = 4$.

Answer

Smallest σ with no false circles (for $k = 4$): $\sigma = 3$. At $\sigma = 1$ the circles are still strong; at $\sigma = 2$ faint arcs remain along the borders; at $\sigma = 2.5$ a faint residue is left at the top and bottom edges; at $\sigma = 3$ none is visible, and the leftover contrast is 0.6% of the range.

What happened in each result. *Keep every 4th pixel:* the rings with local frequency above 0.125 cycles/pixel folded back into the representable band. They reappear as coarse circles around eight extra centers (the points where the folded frequency passes through zero), a pattern that is not in the original at all. The spectrum is the original spectrum copied and overlapped, so energy fills the whole plane. *Blur, then keep every 4th pixel:* the Gaussian removed the energy above f_N first, so there was nothing left to fold. What remains is the center of the pattern, which *is* representable, and flat gray where the rings were too fine.

Why it looks like this

Why $\sigma = 3$ and not the textbook $k/2 = 2$. A Gaussian is not a brick-wall filter: its response falls gradually. With $\sigma = k/2 = 2$ the response at the new Nyquist limit is still 0.29, so 29% of the amplitude of the offending frequencies survives and folds, which is the faint arcs we see. At $\sigma = 3$ it is 0.06. For this image a good rule is therefore $\sigma \approx 0.75k$ rather than $k/2$ (measured on this one image and this one k).

Why not blur even more? Over-blurring removes genuine detail. At $\sigma = 4$ the false circles are gone, but the response at *half* the Nyquist limit is already 0.29, so legitimate inner rings are weakened and the visible pattern shrinks. The best σ is the *smallest* one that removes the aliasing, which is exactly the question the handout asks.

Answer

What this means for a Gaussian pyramid. Every reduction step must blur *before* it subsamples, and the blur has to be matched to the subsampling factor (σ grows in proportion to k). Subsampling an unblurred level would put aliasing into the pyramid, and every coarser level, every Laplacian level built from it and every blend that uses it would inherit it. In practice the pyramid reduces by only a factor 2 per step with a

fixed small kernel, rather than by one large jump.

Why it looks like this

An honest caveat about our own pyramid. The $\frac{1}{16}[1\,4\,6\,4\,1]$ kernel of Section 3 has variance 1 (equivalent to $\sigma = 1$) at $k = 2$, where $f_N = 0.25$. Its response there is $e^{-2\pi^2 \cdot 0.0625} = 0.29$, the same 0.29 as $\sigma = 2$ at $k = 4$. So a single reduce step is not perfectly alias-free; it is the classic compromise (cheap, nearly separable, and small in extent), and what leaks through is small and gets blurred again at the next level.

2.3 A natural image confirms it

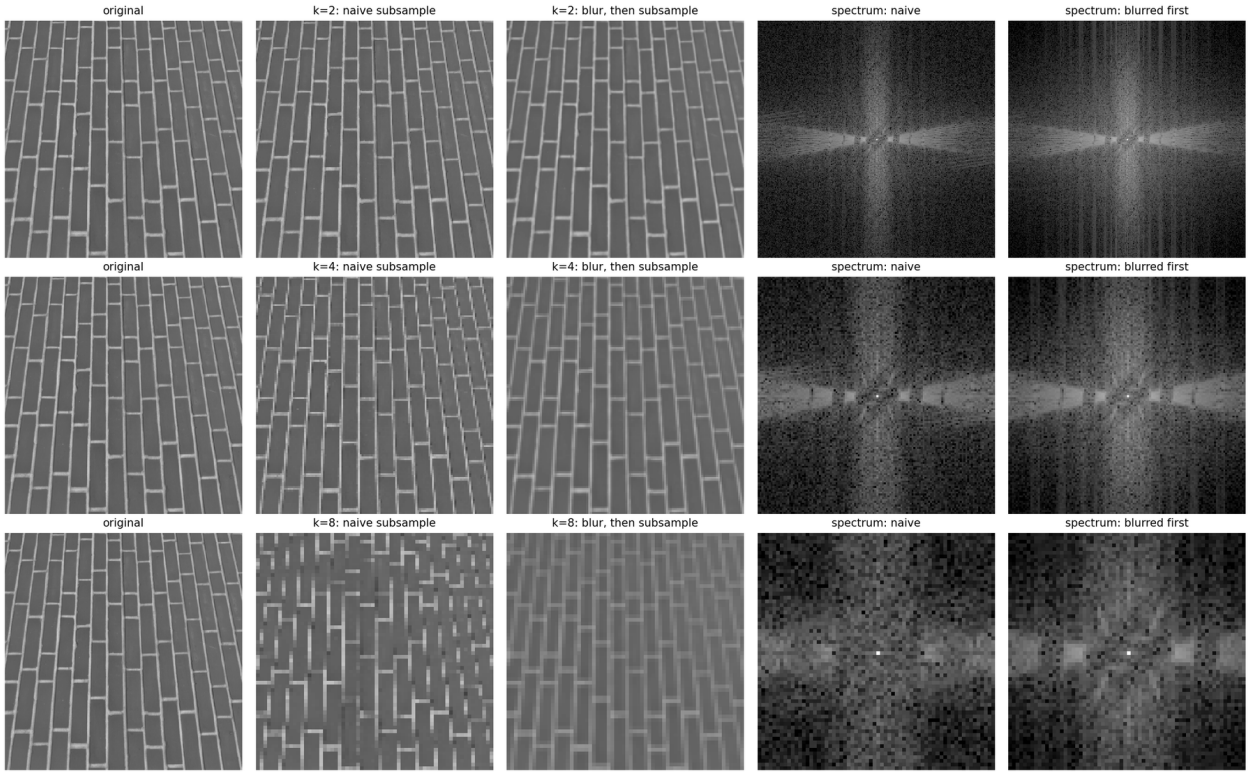


Figure 11: The same experiment on a brick wall (`brick.png`).

Why it looks like this

Bricks are a repeating pattern, the natural-image equivalent of the rings. With naive subsampling at $k = 4$ the courses and the mortar lines become irregular, and at $k = 8$ the wall is scrambled: bricks of different widths and broken lines that do not exist in the original. Blurring first keeps a regular grid of bricks, softer but truthful. Its spectrum has energy only near the center, while the naive spectrum is filled out to the borders: that extra energy is the folded high frequencies.

3 Gaussian and Laplacian pyramids (2.3)

3.1 Implementation

Both `reduce` and `expand` blur with $\frac{1}{16}[1\,4\,6\,4\,1]$ along x and y (our separable convolution, reflect padding).

```
K = outer([1,4,6,4,1], [1,4,6,4,1]) / 256 # separable, rank 1

def reduce(img): # blur, keep every other row/col
   return convolve(img, K, "reflect")[::2, ::2]

def expand(img, out_shape): # zeros between pixels, blur, x4
   out = np.zeros(out_shape + img.shape[2:])
   out[::2, ::2] = img # top-left of each 2x2 block
   return 4.0 * convolve(out, K, "reflect") # odd sides: out_shape drops the extra row
       /col

def laplacian_pyramid(img, levels):
   g = gaussian_pyramid(img, levels) # G0 = img, G(i+1) = reduce(G(i))
   return [g[i] - expand(g[i+1], g[i].shape) for i in range(len(g)-1)] + [g[-1]]

def reconstruct(lap):
   img = lap[-1]
   for L in reversed(lap[:-1]):
      img = L + expand(img, L.shape)
   return img
```

The test image is the facade photograph (466×512, neither side a power of 2). Level sizes, with $\lceil n/2 \rceil$ at each step: 466×512, 233×256, 117×128, 59×64, 30×32, 15×16.



Figure 12: Gaussian pyramid (top) and Laplacian pyramid (bottom, each level stretched to its own contrast, zero = mid-gray); the last Laplacian tile is the coarsest Gaussian level.



Figure 13: Original, image rebuilt from the Laplacian pyramid, and the absolute error stretched to the full range. The maximum error is 1.11×10^{-16}.

Why it looks like this

The stretched error image looks like sparse colored specks, not like an edge map. That is the point: the largest difference is 10^{-16}, about half of double-precision machine epsilon (2.2×10^{-16}), so what we see is rounding noise magnified to full contrast, not a structural error.

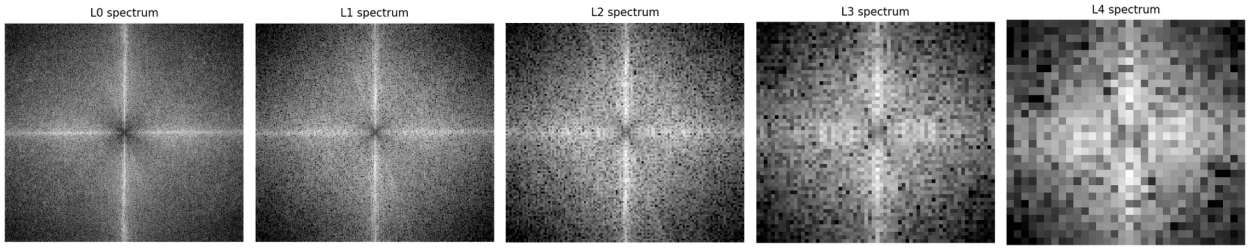


Figure 14: Spectrum of Laplacian levels L_0 to L_4 (each on its own grid). Every level has a dark center, because the low frequencies are exactly what was subtracted.

3.2 Why the rebuild is exact, and why the Lecture 3 version is not

Answer

Exactness. Each Laplacian level is *defined* as $L_i = G_i - \text{expand}(G_{i+1})$, and the rebuild computes $G_i = L_i + \text{expand}(G_{i+1})$ with the *same* `expand` applied to the *same* G_{i+1} (which the rebuild has already recovered). It is the identity $(G_i - E) + E = G_i$ at every level, starting from the stored coarsest level G_{last}, so nothing is approximated; the only error is floating-point rounding. Subsampling really throws information away, but that information is exactly what is stored in L_i: L_i is the residual of the *prediction* $\text{expand}(G_{i+1})$.

The Lecture 3 version is not exact. There each level is the image minus its *blur* at the same size, $G_i - \text{blur}(G_i)$. That works as long as every level keeps the full resolution, so the pieces telescope. As soon as the levels are subsampled to save memory, the rebuild no longer has $\text{blur}(G_i)$: it has only the small image, and $\text{expand}(\text{reduce}(G_i)) \neq \text{blur}(G_i)$ (the former is an interpolation of a subsampled signal). The stored detail was computed against a different prediction than the one used to rebuild, so the sum does not return G_i.

3.3 Total size of the Gaussian pyramid

Answer

The Gaussian pyramid holds $\frac{4}{3}$ of the pixels of the image. Each level has a quarter of the pixels of the one before, so the total is

$$1 + \tfrac{1}{4} + \tfrac{1}{16} + \cdots = \sum_{i \geq 0} 4^{-i} = \frac{1}{1 - 1/4} = \frac{4}{3} \approx 1.333. \tag{4}$$

Our six levels measure 1.3336 (printed by the notebook). The exact partial sum of six terms is 1.3330, and the measured value is slightly larger because odd sizes are rounded *up* ($\lceil n/2 \rceil$), which is also why the rebuild works for odd sides.

Why it looks like this

The 33% overhead is the price of keeping every scale. The Laplacian pyramid has the same number of coefficients, so it is not a compression by itself; what makes it useful is that most of its values are near zero (flat regions have no detail), so it is sparse and compresses well. The factor 4 in `expand` has a

simple reason: three of every four pixels of the zero-padded grid are 0, and the kernel sums to 1, so the blur would divide brightness by 4 (2 per axis). Multiplying by 4 puts it back, and we checked that a flat image stays exactly flat through `expand`, including at odd sizes and at the borders.

4 Multiresolution blending (2.4)

4.1 Method

Given two images A, B and a mask M (1 where we want A), build the Laplacian pyramids L_i^A, L_i^B and the Gaussian pyramid M_i of the mask. At every level the bands are mixed with that level's mask and the result is collapsed exactly as in 2.3:

$$L_i^H = M_i L_i^A + (1 - M_i) L_i^B, \qquad H = \text{reconstruct}\left(L^H\right). \tag{5}$$

The mask given to the function is sharp; its Gaussian pyramid does the softening. Because each level's mask has been reduced i times with a unit-variance kernel, the mask at level i is blurred by $\sigma_i \approx \sqrt{(4^i - 1)/3}$ original pixels:

level i	0	1	2	3	4	5
σ_i (original pixels)	0	1.0	2.2	4.6	9.2	18.5

Fine levels switch images almost abruptly and coarse levels over tens of pixels.

4.2 Blend 1: the oraple

Mask: the left half white, built in code (`vertical_mask`: ones in the left 50% of the columns, zeros elsewhere). Both images are 512×512. **Levels used: 6.** Besides the hard cut required by the handout we also show a single-scale feather (the same mask blurred once with $\sigma = 15$), to show that blurring the mask is not enough on its own.

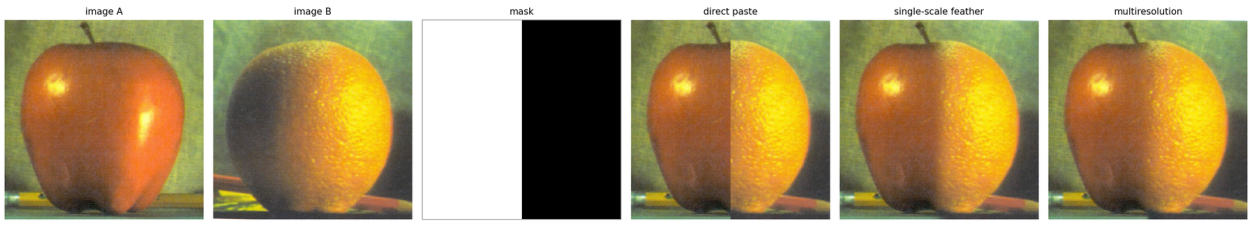


Figure 15: Apple, orange, mask, hard cut, single-scale feather and multiresolution blend (6 levels).

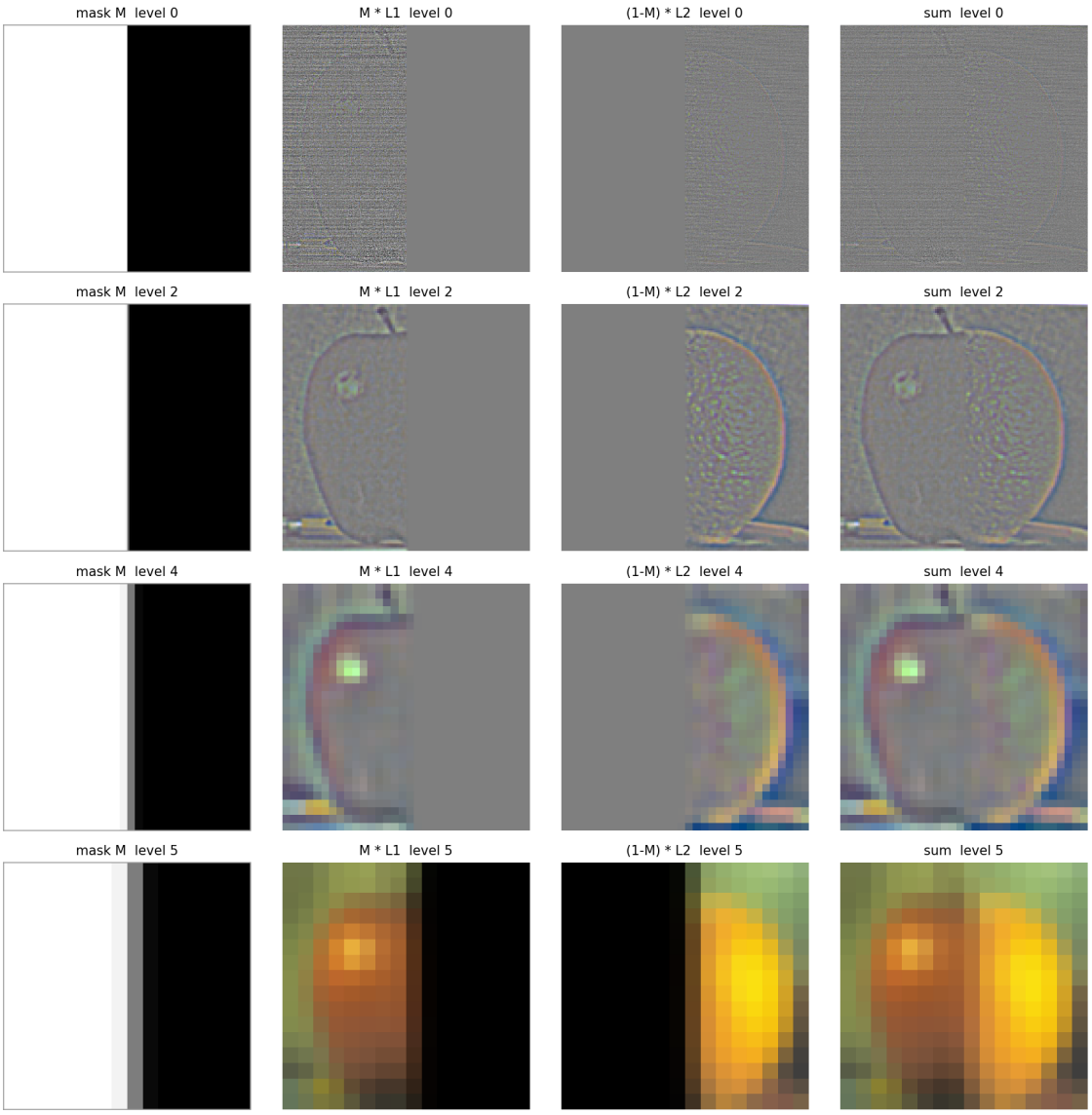


Figure 16: Level by level (levels 0, 2, 4, 5): the mask M_i, $M_i L_i^A$, $(1 - M_i)L_i^B$, and their sum, which is the blended level. Levels 0, 2 and 4 are signed (zero = mid-gray); the last level is the coarsest Gaussian and is shown as is.

Answer

Why blending band by band hides the seam that the hard cut shows. An image is a sum of bands, and each band needs a different transition width to look natural. *Coarse bands* (overall color and brightness) change gently inside each image, so a step in them is conspicuous: they must be mixed over a *wide* region, comparable to their own wavelength. *Fine bands* (the orange's peel, the apple's smooth skin) must be mixed over a *narrow* region, because mixing them over a wide one superimposes two different textures: a ghost, the double exposure of the feather result. A hard cut uses a transition of one pixel for every band, so the coarse bands jump from apple-red to orange-yellow at the cut, and the eye sees that jump as the seam. The mask's Gaussian pyramid gives each band its own width for free: Figure 16 shows the mask nearly sharp at level 0 and spread over tens of pixels at level 5.

Why it looks like this

What the level figure shows. At level 0 ($\sigma_0 = 0$) the mask is a hard edge, and yet there is no visible seam, because that band holds only the finest texture (details of a pixel or two), and a texture is allowed to change abruptly. At level 5 the mask is blurred by about 18 pixels (a transition roughly 45 pixels wide), and the color of the apple fades into the color of the orange gradually. The single-scale feather cannot do

both: with $\sigma = 15$ its transition is wide enough to hide neither the fine texture change nor the color step, and a soft vertical band is still visible in the result.

4.3 Blend 2: a face on a pumpkin

Inputs. A pumpkin photograph (563×1000) and a frontal portrait (595×1000), both resized so that the longer side is 1000 pixels, as the handout recommends.

Alignment (`align_images.py`, two points on each image). The two eye centers of the portrait, $(216, 331)$ and $(381, 331)$, are sent to $(213, 425)$ and $(341, 425)$ on the pumpkin's body. Both pairs are horizontal, so the transform is a pure shrink by 0.776 and a shift (no rotation); the shrink makes the face fit on the pumpkin.

Mask. A 24-point polygon traced on the *portrait* around the skin of the face (hairline to chin, between the ears; hair, ears and neck are left out). The polygon is mapped onto the pumpkin with the *same* two-point transform that moves the pixels, so mask and image cannot drift apart, and it is filled with `cv2.fillPoly`. The result is an irregular, hard-edged shape of 268×366 pixels (14.9% of the image). **Levels used: 5** (the image allows 7).

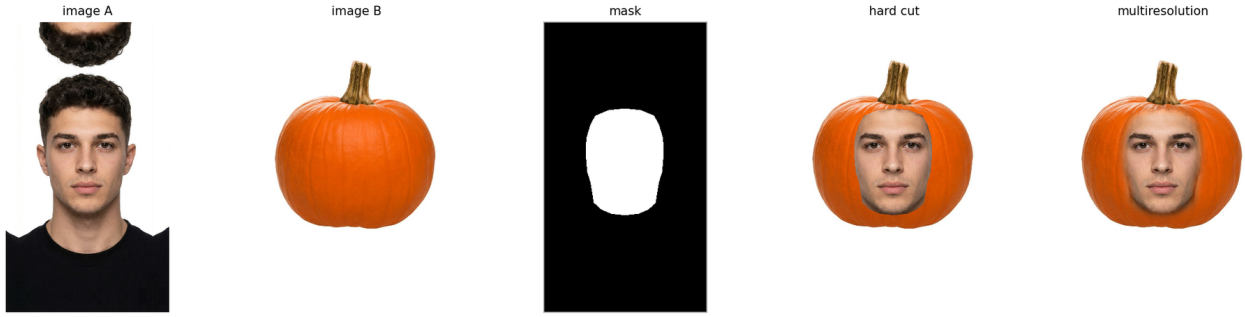


Figure 17: Image A (the portrait, aligned onto the pumpkin's frame), image B (the pumpkin), the mask, the hard cut and the multiresolution blend (5 levels).

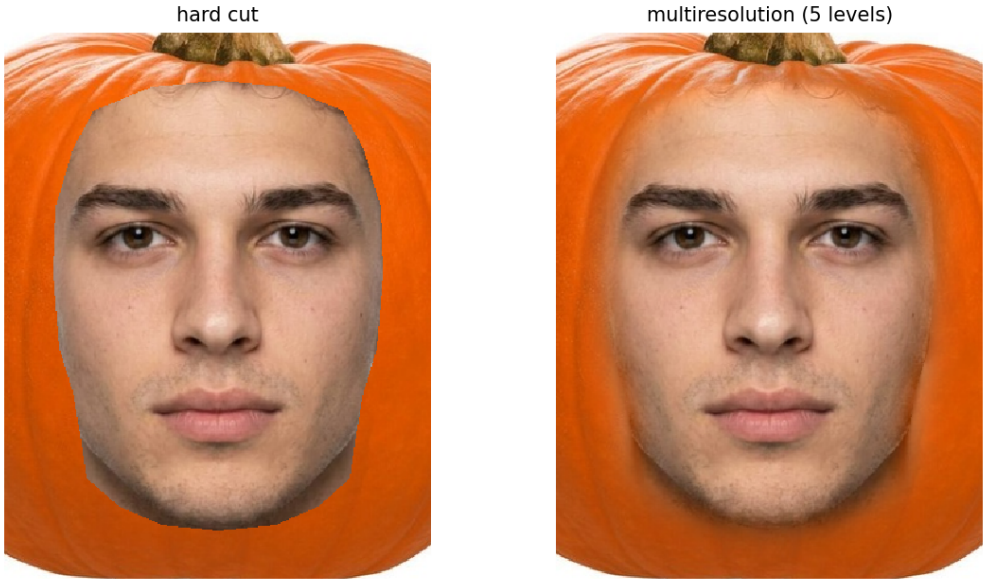


Figure 18: The same result zoomed around the face: hard cut (left) and multiresolution blend (right).

Why it looks like this

Hard cut. The face looks like a sticker: its pale skin stops at a one-pixel outline against the saturated, strongly textured orange, and the eye reads that outline as an edge. **Multiresolution.** The eyes, brows and lips are fine and mid-frequency structure that exists only in the portrait, and inside the polygon the mask is 1 at *every* level, so they stay as crisp as in the original. Only near the outline do the coarse bands (overall color and brightness) fade into the pumpkin's, so the rim of the face takes on a little orange, as if lit by the pumpkin, instead of ending in a line. **What it does not do.** The interior keeps the portrait's own color and lighting (frontal and pale, against a pumpkin lit from the upper left), because inside the mask only the portrait's bands are used. Blending hides the *seam*; it does not make two differently lit objects consistent.

4.4 Blend 3: an eye on the moon

Inputs. A macro photograph of an eye (736×981) and a night photograph of the full moon (667×1000, resized as above). The moon's disc has its center near $(336, 450)$ and a radius of about 214 pixels.

Alignment. The inner and the outer corner of the eye, $(95, 635)$ and $(700, 583)$, are sent to $(170, 464)$ and $(502, 436)$, two points across the moon's disc. The corner-to-corner line has the same tilt in both images, so the rotation is $0.1°$; the scale is 0.549, which makes the eye about 330 pixels wide, roughly 78% of the moon's diameter.

Mask. An 18-point almond-shaped polygon traced on the eye photograph along the tips of the upper lashes and the lower lid, mapped onto the moon through the same transform and filled with `cv2.fillPoly`: 327×170 pixels, 5.8% of the image. **Levels used: 5** (the image allows 7).



Figure 19: Image A (the eye, aligned onto the moon's frame), image B (the moon), the mask, the hard cut and the multiresolution blend (5 levels).

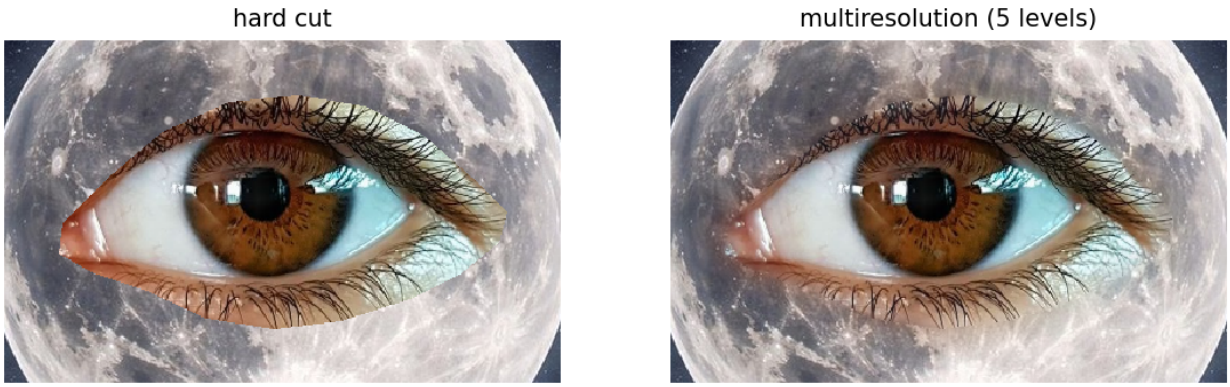


Figure 20: Zoom on the eye: hard cut (left) and multiresolution blend (right).

Why it looks like this

In the hard cut the pasted region keeps a visible rim of pinkish eyelid skin that ends abruptly on the gray moon. In the blend that rim melts into the surface: the lashes and the iris remain sharp, while the skin tone fades out over a few tens of pixels. Because the eye is warm and dark and the moon is cool and light, almost all of the difference between the two images is in the coarse bands, which is exactly where a wide transition is needed.

Favorite: the eye on the moon, level by level. Figure 21 shows four of the five levels (0, 1, 3, 4).

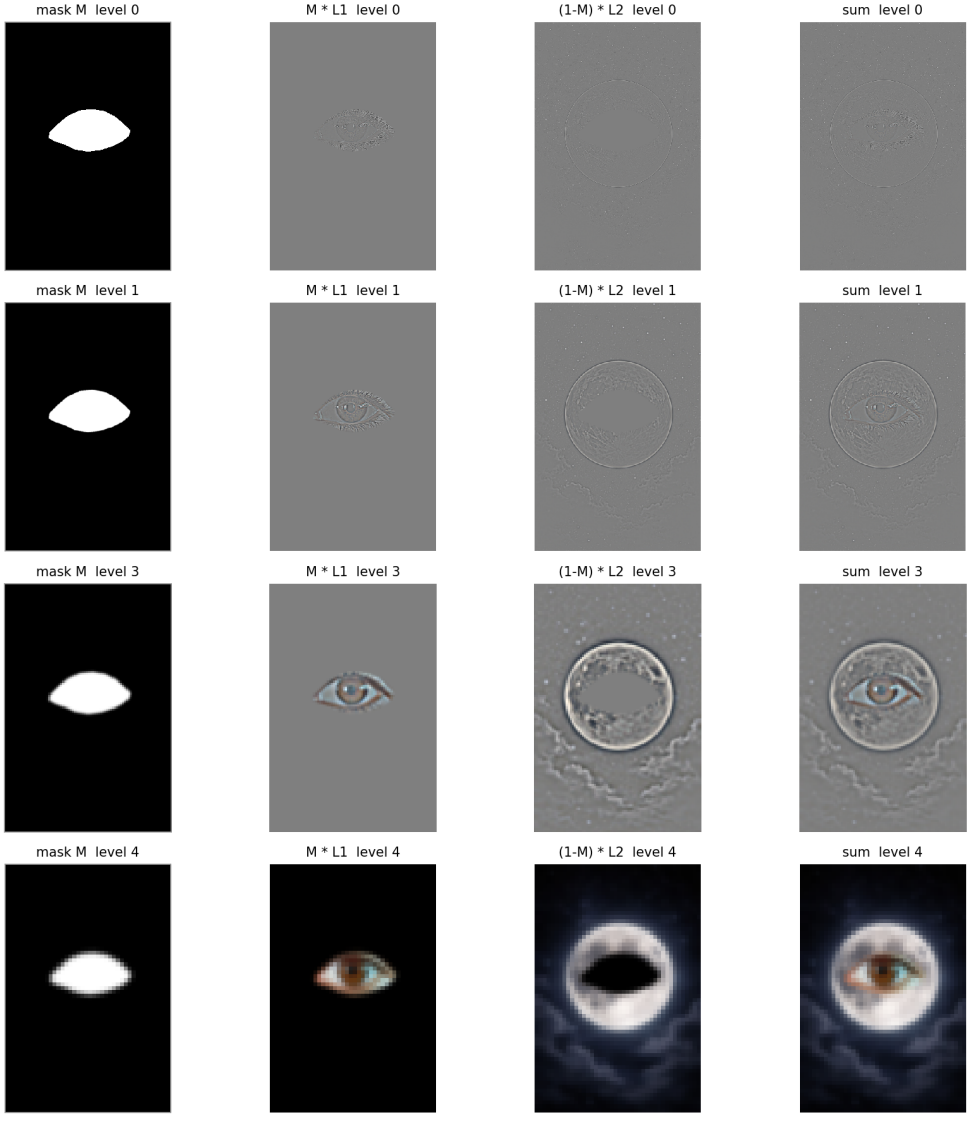


Figure 21: Eye on the moon, levels 0, 1, 3 and 4: the mask M_i, $M_i L_i^A$ (the eye's contribution), $(1 - M_i)L_i^B$ (the moon's contribution) and their sum. Levels 0, 1 and 3 are signed (zero = mid-gray); level 4 is the coarsest Gaussian and is shown as is.

Why it looks like this

Reading the figure. At levels 0 and 1 the eye contributes only thin lashes and edges, and the moon contributes its finest detail (stars, the thin bright rim of the limb); the mask is almost a hard edge, which is harmless because these bands carry no large-scale color. At level 3 the eye appears as soft colored shapes (iris, pinkish corner, white of the eye) and the moon's band has a hole exactly where the mask is 1; the two fill each other, and their sum is continuous. At level 4, the coarsest, the hole in the moon has a soft

edge (the mask is blurred by about 9 pixels), so the moon's glow wraps around the eye gradually. The sum of the two at every level is already the final band, which is why the rebuild shows no seam.

4.5 How many levels?

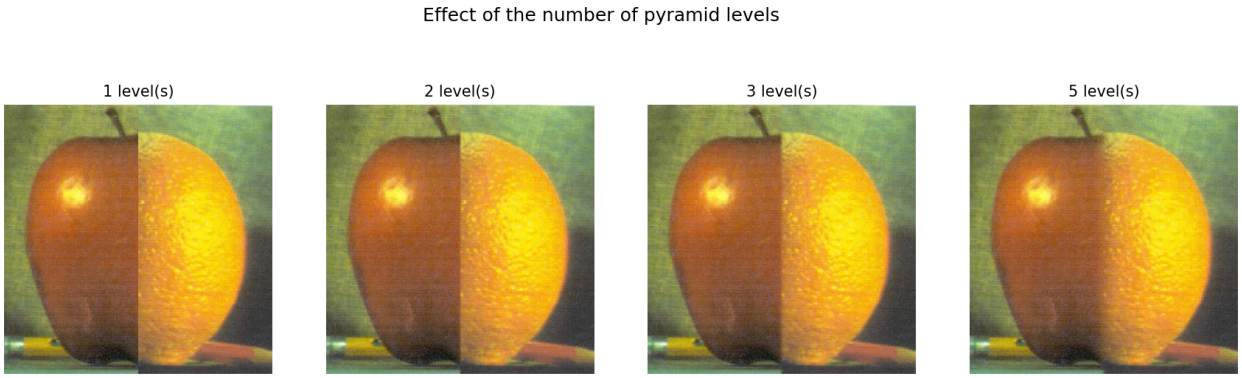


Figure 22: Oraple with 1, 2, 3 and 5 pyramid levels.

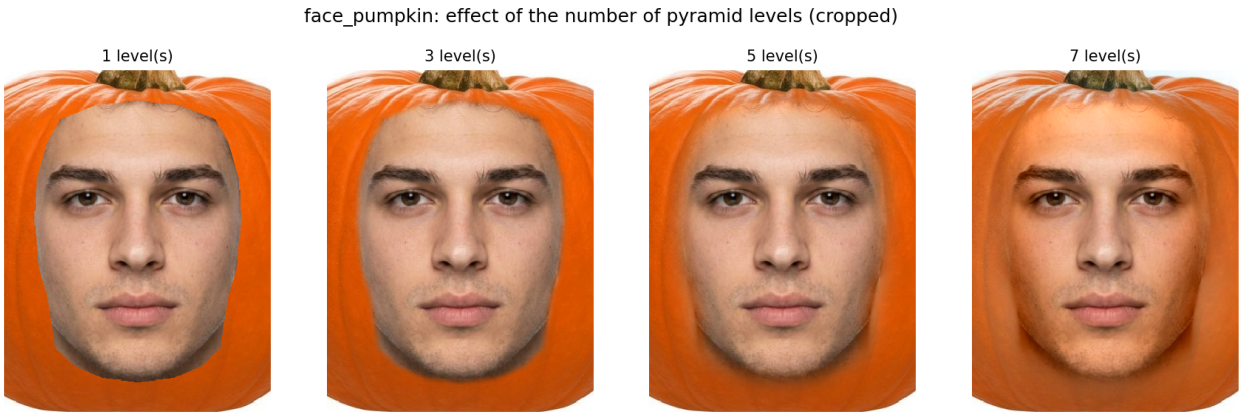


Figure 23: Face on the pumpkin with 1, 3, 5 and 7 levels (cropped around the face).

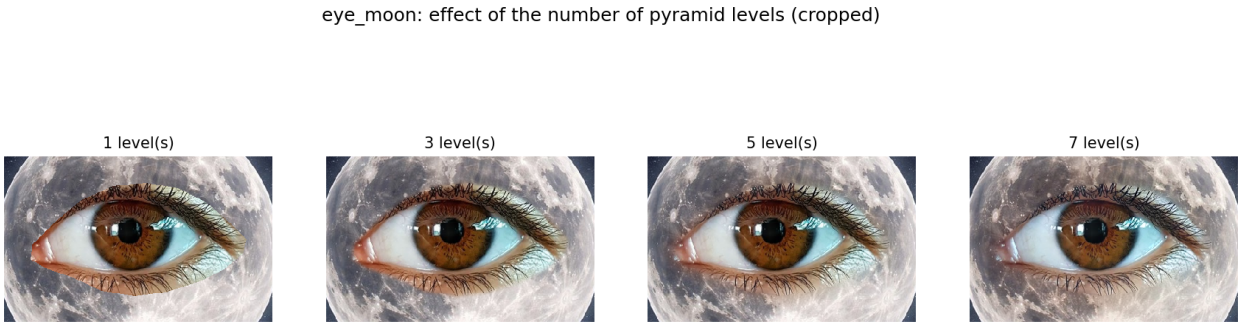


Figure 24: Eye on the moon with 1, 3, 5 and 7 levels (cropped around the eye).

Answer

Levels used. Oraple: 6. Face on the pumpkin: 5. Eye on the moon: 5.
Too few levels leave the seam. One level is the hard cut, since there is no pyramid and only the sharp mask is used. With 2 or 3 levels the widest transition is only a few pixels: the oraple still shows a step in color, the face still has a clear outline, and the eye still has its rim of skin. The seam is gone at 5 levels (we did not render 4).
Too many levels wash out the pasted object. The widest transition grows quickly: the mask at the last level of a L-level pyramid is blurred by σ_{L-1}, which is 9.2 pixels for 5 levels, 18.5 for 6 and 36.9 for 7. At 7 levels the coarsest bands of the face are mixed over about $\pm 2\sigma_6 \approx \pm 74$ pixels from the outline, and the face's half-width is only 134 pixels. Figure 23 shows the result: the cheeks and forehead take on the pumpkin's orange and the central strip (nose, mouth) mostly keeps its own color. The eye on the moon is affected less visibly in our render, but its half-height of 85 pixels is of the same order as the transition at 7 levels, so the same risk applies.

Why it looks like this

A useful rule: choose the *largest* number of levels for which the widest transition ($\approx 2\sigma_{L-1}$ on each side of the outline) stays well inside the pasted object. The oraple can afford 6 levels because the object is half of the image (256 pixels wide), whereas the face (268 pixels across) and the eye (170 pixels high) are smaller, so 5 is the safe choice. Fewer levels would sharpen the outline; more would dissolve the object. The pyramid's depth is the knob that trades *seam visibility* against *object identity*.

5 Conclusions

All four parts of this report are statements about *frequency*:

- A hybrid image works because the visual system reads different frequency bands at different distances, so two images can share one picture if they live in different bands (2.1).
- Aliasing is what happens when that band structure is ignored while sampling; the cure is to remove the band you cannot represent *before* sampling, and to do it strongly enough: a Gaussian needed $\sigma \approx 0.75k$ here, not $k/2$ (2.2).
- The Laplacian pyramid is a bookkeeping of the bands that blur-and-subsample discards, and it is exact because the same `expand` is used to make and to undo the prediction (2.3).
- Multiresolution blending gives every band a transition width proportional to its own scale, which a single cut or a single feather cannot do (2.4).

A Code organization and checks

File	Content
`hybrid.py`	`make_hybrid`, low/high-pass, `log_fft`
`aliasing.py`	naive and anti-aliased subsampling, test signals, `outer_ring_std`
`pyramids.py`	`reduce`, `expand`, Gaussian and Laplacian pyramids, `reconstruct`, `total_size_ratio`
`blending.py`	`multires_blend`, masks (vertical, circle, ellipse, polygon), hard cut and feather baselines
`viz.py`	every figure in this report
`align_images.py`	starter-pack aligner (two-point similarity transform)
`test_part2.py`	15 tests: exact reconstruction (even, odd, color), kernel weights, flat images through `expand`, the $4/3$ pyramid size, anti-aliasing, blend limits and per-level sums